

Query-Conditioned Test-Time Self-Training for Large Language Models

Chaehee Song^{*1} Minseok Seo^{*1} Yeeun Seong² Doyi Kim² Changick Kim^{†1}

¹School of Electrical Engineering, KAIST

²Graduate School of Green Growth and Sustainability, KAIST

{chaehee.song, minseok.seo, yeeunseong, doyi.kim, changick}@kaist.ac.kr

^{*}Equal contribution. [†]Corresponding author.

Abstract

Large language models (LLMs) are typically deployed with fixed parameters, and their performance is often improved by allocating more computation at inference time. While such test-time scaling can be effective, it cannot correct model misconceptions or adapt the model to the specific structure of an individual query. Test-time optimization addresses this limitation by enabling parameter updates during inference, but existing approaches either rely on external data or optimize generic self-supervised objectives that lack query-specific alignment. In this work, we propose Query-Conditioned Test-Time Self-Training (QueST), a framework that adapts model parameters during inference using supervision derived directly from the input query. Our key insight is that the input query itself encodes latent signals sufficient for constructing structurally related problem–solution pairs. Based on this, QueST generates such query-conditioned pairs and uses them as supervision for parameter-efficient fine-tuning at test time. The adapted model is then used to produce the final answer, enabling query-specific adaptation without any external data. Across seven mathematical reasoning benchmarks and the GPQA-Diamond scientific reasoning benchmark, QueST consistently outperforms strong test-time optimization baselines. These results demonstrate that query-conditioned self-training is an effective and practical paradigm for test-time adaptation in LLMs. Code is available at <https://github.com/chssong/Query-Conditioned-TTST>.

1 Introduction

The success of large language models (LLMs) [38, 4, 22, 30, 1, 31, 11, 35] is driven by massive data and large-scale computation. While continuously retraining and redeploying models is the most direct way to improve performance, it incurs prohibitive costs and is often impractical in real-world settings. As a practical alternative, performance is typically improved by increasing test-time computation [28, 21, 29] or refining prompts [37, 12, 42]. However, user queries vary widely and demand different reasoning strategies, which makes adapting models to individual queries difficult through these approaches alone.

To address this limitation, recent work [41, 3, 14, 9, 5] has explored test-time optimization (TTO), which adapts model parameters at inference time to better align with a given query. Unlike test-time scaling, which leaves model parameters fixed, TTO modifies them directly, enabling more flexible and input-dependent behavior.

For example, TTT-NN [5] improves performance by retrieving similar problems from an external database and fine-tuning the model on them. However, such approaches rely on large-scale data storage and retrieval, which introduces practical limitations. In contrast, TLM [9] updates model

Table 1: Comparison of test-time improvement strategies for large language models across four key properties. While existing methods satisfy these properties only partially, QueST satisfies all of them, enabling effective and query-specific adaptation at inference time.

Method	Query-specific	External data-free	Test-time update	Input-conditioned signal
Test-time scaling	✗	✓	✗	✗
Self-evolving	✗	✓	✗	✗
TTT-NN (ICLR’24)	✓	✗	✓	✓
TLM (ICML’25)	✗	✓	✓	✓
QueST	✓	✓	✓	✓

parameters by minimizing the perplexity of the input itself, but this objective is not directly aligned with the specific reasoning structure required by individual queries.

Despite these limitations, two important observations emerge from prior work. First, model performance improves when it is adapted using problems that are structurally similar to the input query. Second, the input query itself contains latent signals that can guide effective adaptation. These observations suggest that it is possible to construct meaningful supervision directly from the input query, without relying on external data, enabling more efficient test-time adaptation.

In this work, we propose Query-Conditioned Test-Time Self-Training (QueST), a framework that leverages this insight. Given a query, QueST treats it as a seed and generates structurally related problem–solution pairs conditioned on the input. These generated samples are then used to perform lightweight LoRA-based updates at test time. The adapted model is subsequently used to produce the final response, enabling dynamic and query-specific adaptation.

Beyond TTO, self-evolving methods [10, 17] also improve performance by generating and solving problems. While QueST shares this generate-and-solve intuition, it differs fundamentally in approach. Self-evolving methods typically rely on global data generation and carefully engineered prompts, whereas QueST performs localized adaptation directly conditioned on the input query, without requiring external data or complex prompt design.

These differences are summarized in Table 1. Existing approaches satisfy only a subset of the properties required for effective test-time adaptation. For instance, test-time scaling and self-evolving methods do not perform per-query parameter updates at test time, and therefore cannot directly specialize model parameters to individual input queries. While both TTT-NN and TLM leverage input-conditioned signals, TTT-NN relies on external data and TLM lacks explicit query-specific alignment.

In contrast, QueST is the only approach that satisfies all four key properties: query-specific adaptation, independence from external data, test-time parameter updates, and input-conditioned signal. This enables QueST to provide a practical and effective framework for adapting large language models to individual queries at inference time.

We evaluate QueST on seven mathematical reasoning benchmarks, where it achieves state-of-the-art performance and improves average accuracy by 6.44 percentage points over the corresponding base models. We further evaluate QueST on the GPQA-Diamond scientific reasoning benchmark, showing that QueST generalizes beyond mathematical reasoning.

2 Related work

2.1 Test-time Scaling

Test-time scaling (TTS) [21, 28] improves model performance by allocating additional computation during inference. Early approaches are rooted in Chain-of-Thought (CoT) prompting [37], which enables step-by-step reasoning at inference time. Building on this, subsequent methods improve performance by sampling multiple reasoning trajectories [34] and aggregating them via self-consistency [33], or by selecting the best candidate from multiple outputs using Best-of-N strategies [2]. More recent approaches further enhance TTS by incorporating outcome verifiers and process reward models (PRMs) [16], which evaluate final answers or intermediate reasoning steps,

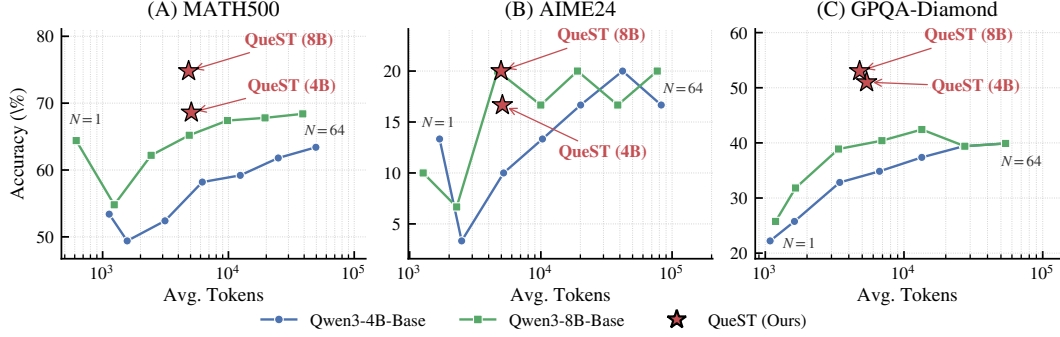


Figure 1: Comparison of token usage and performance across three benchmarks: (a) MATH500 (high-school level mathematics), (b) AIME24 (advanced mathematical reasoning), and (c) GPQA-Diamond (general scientific reasoning). Each point on the baseline curves corresponds to sampling-based TTS using self-consistency decoding with $N \in \{1, 2, 4, 8, 16, 32, 64\}$ samples. Star markers denote QueST. Across all three benchmarks, QueST achieves superior accuracy while using substantially fewer test-time tokens.

often combined with search strategies such as beam search or tree search [39]. In addition, iterative refinement methods, such as Self-Refine [18] and Reflexion [25], improve performance through repeated self-correction and feedback.

Despite these advances, TTS methods operate without updating model parameters, relying solely on increasingly sophisticated sampling, search, and self-correction procedures at inference time. As a result, their effectiveness is fundamentally bounded by the capabilities already encoded in the base model, limiting their ability to correct underlying misconceptions or directly adapt model parameters to the specific structure of individual queries. Moreover, TTS methods incur substantial token costs. As shown in Figure 1, QueST achieves higher performance than sampling-based TTS baselines while requiring fewer tokens.

2.2 Self-evolving

Self-evolving methods aim to reduce reliance on expensive human-curated data by enabling models to generate tasks and improve through accumulated experience. This line of work is inspired by self-play in reinforcement learning, where AlphaGo [26] combined human demonstrations with self-play, while AlphaZero [27] achieved superhuman performance without human-annotated training data.

Recent efforts have extended similar ideas in LLMs. R-Zero [10] proposes a co-evolution framework in which challenger and solver models generate and learn from their own tasks, while SPICE [17] extends this paradigm through corpus-grounded self-play using external document collections. Despite its effectiveness, SPICE remains tied to human-produced corpora, indicating that fully self-driven improvement in LLMs is still an open direction.

Unlike board games, natural language tasks often lack a well-defined environment or reliable automatic verification signals. As a result, self-evolving LLMs often face fundamental challenges in maintaining data quality, training stability, and consistent learning dynamics.

While QueST shares the intuition of generating and solving tasks for self-improvement, it fundamentally differs from prior self-evolving approaches. Existing self-evolving methods operate in an offline setting, where models iteratively accumulate and learn from generated data. In contrast, QueST performs *online, query-conditioned adaptation* by generating task-specific QA pairs directly from a given user query at test time, enabling immediate and targeted model adaptation.

2.3 Test-time Optimization

Test-time optimization (TTO) [32, 15, 24] updates model parameters during inference using signals available at test time, thereby specializing the model to the test input or target distribution. This paradigm has been extensively studied in computer vision, where performance degradation is often

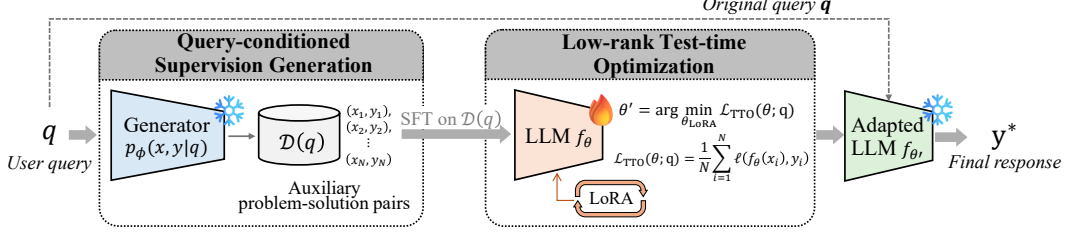


Figure 2: Overview of QueST. Given a user query q , QueST first generates query-conditioned auxiliary problem–solution pairs $\mathcal{D}(q)$ that reflect the underlying reasoning patterns of the query. These generated samples serve as supervision for low-rank test-time optimization via LoRA, enabling efficient parameter adaptation at inference time. The adapted model then produces the final response y^* , achieving query-specific adaptation without relying on external data.

attributed to input distribution shifts. Accordingly, most methods adapt a pretrained model by exploiting auxiliary signals available at inference time.

A representative approach in vision is TENT [32], which updates model parameters by minimizing prediction entropy at test time, typically by adapting only normalization layers.

Recently, TTO has been extended to large language models. For example, TTT-NN [5] retrieves similar samples from a large external corpus conditioned on the input query and fine-tunes the model before prediction, making its adaptation input-conditioned but dependent on external data. TLM [9] adapts LLMs using unlabeled test data via input perplexity minimization, and is primarily designed for domain-level adaptation under distribution shift, rather than explicitly targeting query-specific optimization.

In contrast, QueST performs test-time optimization using supervision generated directly from the input query, achieving both query-specific adaptation and independence from external data, two properties that prior TTO methods satisfy only partially. Moreover, QueST enables efficient and targeted adaptation at the level of individual queries through parameter-efficient updates.

3 Method

3.1 Problem Definition

Given a pre-trained language model f_θ and a user query q , the goal is to generate an accurate response y . In standard inference, the model directly predicts $y \sim p_\theta(y | q)$ without any adaptation.

However, real-world queries often follow a distribution that differs from the pre-training distribution, leading to suboptimal performance on domain-specific or challenging inputs. Ideally, we would like to adapt the model to the local distribution induced by the query q .

Test-time optimization (TTO) addresses this by updating model parameters at inference time, and then using the adapted parameters θ' to generate the final response:

$$\theta' = \theta - \eta \nabla_\theta \mathcal{L}_{\text{TTO}}(\theta; q). \quad (1)$$

The key challenge lies in defining an informative and reliable test-time objective $\mathcal{L}_{\text{TTO}}$, since natural language queries do not provide explicit labels or structured constraints. Existing approaches either rely on external data or optimize generic self-supervised objectives, both of which provide weak or misaligned supervision for the reasoning structure of the current query.

QueST addresses this by constructing query-conditioned supervision $\mathcal{D}(q)$ from the input query and using it to define the test-time objective $\mathcal{L}_{\text{TTO}}$.

3.2 Query-conditioned Supervision Generation

We assume that q is sampled from an underlying local data distribution $\mathcal{D}_q$, which consists of problems sharing similar concepts, assumptions, and reasoning patterns. Under this assumption,

Algorithm 1 Query-Conditioned Test-Time Self-Training (QueST)

Require: Pre-trained model f_θ , query q
1: Generate query-conditioned dataset $\mathcal{D}(q) = \{(x_i, y_i)\}_{i=1}^N$
2: Initialize LoRA parameters θ_{LoRA}
3: **for** $t = 1$ to T **do**
4: Sample $(x_i, y_i) \sim \mathcal{D}(q)$
5: Update θ_{LoRA} by minimizing $\ell(f_\theta(x_i), y_i)$
6: **end for**
7: Obtain adapted model $f_{\theta'}$
8: Generate final answer $y^* \sim p_{\theta'}(y \mid q)$
9: **return** y^*

solving q can be viewed as generalizing within $\mathcal{D}_q$. Therefore, optimizing the model on samples that approximate $\mathcal{D}_q$ can improve its ability to solve the original query q .

Existing approaches either retrieve samples from an external dataset $\mathcal{D}_{\text{ext}}$, or generate pseudo-labels using the model itself:

$$(x_i, y_i) \sim p_\theta(x, y), \quad (2)$$

both of which are independent of the specific query q and often lead to weak or misaligned supervision.

In contrast, QueST generates a dataset $\mathcal{D}(q)$ consisting of auxiliary problem–solution pairs conditioned on q :

$$\mathcal{D}(q) = \{(x_i, y_i)\}_{i=1}^N, \quad (x_i, y_i) \sim p_\phi(x, y \mid q), \quad (3)$$

where p_ϕ denotes the auxiliary problem generator. In practice, we use the base model itself as the generator. Each generated pair shares core concepts and reasoning patterns with the original query, so that $\mathcal{D}(q)$ serves as an empirical approximation of $\mathcal{D}_q$ and provides query-aligned supervision.

Using this dataset, we define the test-time objective as

$$\mathcal{L}_{\text{TTO}}(\theta; q) = \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(x_i), y_i), \quad (4)$$

where ℓ denotes the cross-entropy loss; minimizing this objective adapts the model to the local distribution around q .

3.3 Low-rank Test-time Optimization

To adapt the model efficiently at test time, we employ parameter-efficient fine-tuning using low-rank adaptation (LoRA) [8]. Full fine-tuning is computationally expensive and unstable in a test-time setting, especially when only a small number of supervision samples are available. Therefore, we restrict updates to a low-rank parameterization.

Given the query-conditioned dataset $\mathcal{D}(q)$, we optimize the model parameters by minimizing the following objective:

$$\theta' = \arg \min_{\theta_{\text{LoRA}}} \mathcal{L}_{\text{TTO}}(\theta; q), \quad (5)$$

where only θ_{LoRA} is updated and θ remains frozen, yielding the adapted parameters θ' .

We perform T optimization steps using standard supervised fine-tuning (SFT) on $\mathcal{D}(q)$. This lightweight adaptation is sufficient to capture the local structure of the query-conditioned distribution while maintaining computational efficiency and stability. After optimization, the adapted model $f_{\theta'}$ generates the final response $y^* \sim p_{\theta'}(y \mid q)$ for the original query q .

An overview of the QueST framework is illustrated in Figure 2, and the complete procedure is summarized in Algorithm 1.

4 Experiments

4.1 Experimental Setup

We evaluate QueST to assess whether query-conditioned supervision provides stronger adaptation signals than the alternative test-time objectives used by prior test-time optimization methods. Our

Table 2: Quantitative comparison of QueST with existing test-time optimization methods across seven mathematical benchmarks. All experiments are conducted under the same experimental setup, following the official implementations of each method.

Model Name	Average	AMC	Minerva	MATH	GSM8K	Olympiad	AIME25	AIME24
<i>Qwen3-4B</i>								
Base Model	36.59	45.00	21.32	55.20	91.05	23.56	6.67	13.33
TENT	36.69	47.50	21.69	54.20	91.05	22.37	10.00	10.00
TLM	36.77	45.00	19.49	53.60	90.67	21.93	6.67	20.00
QueST (ours)	42.58	52.50	23.10	70.80	92.10	26.20	13.33	20.00
<i>Qwen3-8B</i>								
Base Model	24.66	27.50	13.24	42.60	75.66	13.63	0.00	0.00
TENT	23.86	25.00	14.71	40.20	74.37	12.74	0.00	0.00
TLM	26.28	32.50	12.13	37.40	74.98	13.63	6.67	6.67
QueST (ours)	31.44	35.00	15.80	54.60	78.20	16.50	6.67	13.33
<i>Qwen3-4B-Base</i>								
Base Model	34.53	35.00	22.06	53.40	90.14	24.44	3.33	13.33
TENT	36.61	47.50	23.16	53.00	88.78	23.85	10.00	10.00
TLM	36.20	42.50	22.43	54.60	89.31	24.59	10.00	10.00
QueST (ours)	41.74	50.00	24.10	68.60	91.66	27.80	13.33	16.66
<i>Qwen3-8B-Base</i>								
Base Model	42.00	57.50	26.84	64.40	92.65	32.59	10.00	10.00
TENT	41.59	52.50	26.47	65.00	92.27	31.56	13.33	10.00
TLM	42.58	62.50	25.74	65.40	91.81	32.59	10.00	10.00
QueST (ours)	47.77	65.00	28.90	74.80	93.20	35.80	16.67	20.00
<i>OctoThinker-3B</i>								
Base Model	21.74	25.00	11.76	36.60	60.27	11.85	0.00	6.67
TENT	20.14	17.50	11.40	37.60	59.59	11.56	0.00	3.33
TLM	22.04	20.00	12.50	38.80	60.80	12.15	3.33	6.67
QueST (ours)	28.59	27.50	19.12	46.40	67.85	19.26	10.00	10.00
<i>OctoThinker-8B</i>								
Base Model	28.05	25.00	13.97	45.80	79.91	18.37	3.33	10.00
TENT	27.32	25.00	13.60	45.20	79.38	18.07	3.33	6.67
TLM	28.76	27.50	14.71	46.60	80.52	18.67	3.33	10.00
QueST (ours)	34.11	35.00	19.49	52.00	84.53	24.44	10.00	13.33

comparisons include TENT [32], a widely used test-time adaptation method originally developed for computer vision, and TLM [9], a recent test-time learning approach for large language models.

We conduct experiments on multiple model architectures to demonstrate the generality of our method. Specifically, we use Qwen3-4B-Base and Qwen3-8B-Base as representative base LLMs, Qwen3-4B and Qwen3-8B as the corresponding post-trained variants, and OctoThinker-3B and OctoThinker-8B [36] as reasoning-oriented models. For post-trained Qwen3 models, all evaluations are performed in non-thinking mode, which is the model’s default inference mode without explicit reasoning chains, to isolate the effect of QueST from the model’s built-in reasoning traces.

We evaluate on a diverse set of benchmarks covering both mathematical and scientific reasoning. For mathematical reasoning, we use seven datasets: AMC [20], Minerva [13], MATH500 [7, 16], GSM8K [2], OlympiadBench [6], AIME25 [19], and AIME24 [19]. To assess generality beyond mathematical reasoning, we additionally evaluate on GPQA-Diamond [23], a challenging benchmark for graduate-level scientific reasoning.

Question (Easy)
Q. Simplify $(-k + 4) + (-2 + 3k)$.
Generated Question
Q. Simplify $(9 - 4x) + (6x - 11)$.
Question (Hard)
Q. A hexagon is inscribed in a circle: [asy] pair pA, pB, pC, pD, pE, pF, pO; pO = (0, 0); pA = pO + dir(-10); pB = pO + dir(60); pC = pO + dir(130); pD = pO + dir(170); pE = pO + dir(-160); pF = pO + dir(-80); draw(pA--pB--pC--pD--pE--pF--pA); label("\$105^\circ\$", pF, N * 2); label("\$110^\circ\$", pB, SW * 1.5); label("\$\alpha\$", pD, E); draw(circle(pO, 1)); [/asy] What is the measure of α , in degrees?
Generated Question
Q. A pentagon is inscribed in a circle with vertices in order A, B, C, D, E . The interior angles at B and E are 118° and 102° , respectively. If the interior angle at D is θ , what is the value of θ in degrees?

Figure 3: Qualitative examples of problems generated by QueST from input queries. The top row shows a simple surface-level variation, while the bottom row presents a more challenging structural transformation, demonstrating the range of generated tasks.

4.2 Implementation Details

All experiments are conducted on a single NVIDIA H200 GPU. For QueST, we use LoRA [8] with rank 16 and perform T optimization steps using AdamW with a learning rate of 1×10^{-4} . We set $T = 10$ for most benchmarks and $T = 40$ for AIME24 and AIME25, which require longer reasoning. For each query, we generate $N=5$ auxiliary problem-solution pairs for test-time adaptation. The optimization objective is the standard SFT loss, i.e., cross-entropy between the model predictions and the corresponding generated solution tokens for each auxiliary problem.

For fair comparison, all methods use the same system prompt: *“Please reason step by step, and put your final answer within `\boxed{\}`. Carefully reconsider the underlying concept of the question.”* Additional implementation details are provided in Appendix A.

4.3 Mathematical Benchmark Results

Table 2 presents the performance of QueST across seven mathematical benchmarks, compared with existing TTO methods. We exclude TTT-NN [5] from comparison, as it relies on retrieving similar samples from a large external corpus, which cannot be fairly reproduced under our experimental setup due to the lack of access to the original retrieval database.

Across most datasets, prior TTO methods such as TENT and TLM show only marginal improvements, or even performance degradation. This suggests that entropy minimization and perplexity reduction provide weak or misaligned signals for structured, multi-step mathematical reasoning.

In contrast, QueST consistently achieves the largest performance gains across all benchmarks and model configurations, with average improvements of +5.77 to +7.21 percentage points over the base models, establishing state-of-the-art results. This demonstrates that combining query-conditioned supervision with test-time optimization is particularly effective for reasoning-intensive tasks, where adapting the model to the specific problem structure is critical.

4.4 Qualitative Analysis

Query-conditioned Problem Generation. Figure 3 presents qualitative examples of problems generated by QueST from input queries, with varying levels of transformation. The top example preserves the algebraic structure of the original question, with only minor changes in coefficients and variables. In contrast, the bottom example reformulates a diagram-based hexagon problem into a symbolic pentagon problem with different reasoning requirements.

Question 150 (MATH500)	
For every positive integer n, let $\text{mod}_5(n)$ be the remainder obtained when n is divided by 5. Define a function $f: \{0,1,2,3,\dots\} \times \{0,1,2,3,4\} \rightarrow \{0,1,2,3,4\}$ recursively as follows: $f(i,j) = \begin{cases} \text{mod}_5(j+1) & \text{if } i = 0 \text{ and } 0 \leq j \leq 4 \\ f(i-1,1) & \text{if } i \geq 1 \text{ and } j = 0 \\ f(i-1, f(i,j-1)) & \text{if } i \geq 1 \text{ and } 1 \leq j \leq 4 \end{cases}$ What is $f(2015,2)$?	
Baseline Answer	QueST Answer
To determine $f(2015, 2)$, we need to understand the recursive definition of the function f. The function f is defined as follows: $[\dots]$ We observe that the values of $f(i, j)$ for $i = 2$ and $j = 0, 1, 2, 3, 4$ are periodic with a period of 5. Specifically, the values repeat every 5 steps. Therefore, for $i = 2015$, we can reduce $2015 \bmod 5$ to find the equivalent i in the periodic sequence. $[\dots]$ $2015 \bmod 5 = 0$ So, $f(2015, 2) = f(0, 2) = 3$. Thus, the final answer is: boxed{3}	For $i=0$, $f(0,j) = \text{mod}_5(j+1)$. $f(0,0)=1, \dots, f(0,4)=0$. $[\dots]$ $g_i(0)=g_{i-1}(1)$ and $g_i(j)=g_{i-1}(g_{i-1}(j))$. For $i=1$: $f(1,j) \equiv j+2 \pmod 5$. $[\dots]$ Claim: $f(i,j) \equiv aj+b \pmod 5$. $[\dots]$ For $i=2015$, $a \equiv 2^{2016} - 1 \pmod 5$ and $b \equiv 2^{2016} \pmod 5$. $f(2015,j) \equiv 0 \cdot j + 1 = 1 \pmod 5$. Final Answer: boxed{1}

Figure 4: An example demonstrating the effectiveness of QueST in correcting erroneous predictions. While the baseline model fails to produce the correct answer, QueST adapts at test time using query-conditioned supervision and successfully recovers the correct solution.

These examples indicate that QueST can generate problems ranging from surface-level variations to structural transformations depending on the input query. Consistent with prior work showing the diversity of self-generated problems [40, 10, 35], QueST further leverages these generated problems as supervision signals for test-time adaptation.

Error Correction. Figure 4 presents a case where QueST corrects an incorrect prediction by leveraging problems generated from the user query. The base model fails to reflect the recursive structure of the function and instead relies on a simplified periodicity assumption, leading to an incorrect answer. This reflects a common failure mode where language models depend on superficial patterns rather than underlying structure.

In contrast, QueST generates additional problems from the user query and uses them as auxiliary information at test time. By considering multiple input–output examples with similar structure, the model’s prediction shifts toward patterns more consistent with the problem, ultimately producing the correct answer. This example shows that QueST does more than refine output formatting; it can alter the model’s test-time behavior in a way that better aligns with the structure of the query.

4.5 Ablation Study

Table 3 presents ablation and cross-domain analysis for QueST.

Component Contributions. The left table evaluates the contribution of query conditioning, LoRA adaptation, and self-generated QA. As shown, self-generated QA without query conditioning yields limited performance, and adding LoRA alone provides only marginal improvement. In contrast, query-conditioned QA substantially improves accuracy, with the best result obtained when combined with LoRA. This indicates that query-specific supervision is critical for effective test-time adaptation.

Number of Generated Problems. The middle table analyzes the effect of the number of generated problems. Performance consistently improves as the number of generated problems increases, with the best result obtained using $N = 5$. This suggests that additional query-conditioned supervision provides more useful adaptation signals for test-time optimization. Thus, we use $N = 5$ in the main experiments, balancing performance and computational cost. Extended ablation with larger N is provided in Appendix D.

Table 3: Ablation and analysis of QueST. (Left) Contribution of query conditioning, LoRA adaptation, and self-generated QA on Qwen3-4B-Base using MATH500. (Middle) Effect of the number of generated problems used during test-time optimization on Qwen3-4B-Base, evaluated on MATH500. (Right) QueST applied to Qwen3-4B-Base and Qwen3-4B on GPQA-Diamond.

Query	LoRA	Self-QA	Acc	#Problems	Acc	Model	Acc
×	×	✓	58.00	1	57.80	Qwen3-4B-Base	22.22
×	✓	✓	58.80	3	63.00	Qwen3-4B-Base + QueST	46.97
✓	×	✓	66.60	4	64.20	Qwen3-4B	32.32
✓	✓	✓	68.60	5	68.60	Qwen3-4B + QueST	51.01

Cross-domain Generalization. We evaluate QueST on the scientific reasoning benchmark GPQA-Diamond. QueST consistently improves performance across both base and post-trained Qwen3-4B models. These results suggest that QueST generalizes beyond mathematical reasoning tasks and is effective across different domains.

5 Limitations and Future Work

QueST relies on supervision signals derived from user queries, and its effectiveness depends on the quality and informativeness of the input. When queries are ambiguous, underspecified, or contain incorrect assumptions, the generated supervision may be weak or misleading, which can limit the effectiveness of test-time adaptation.

As future work, one direction is to move beyond per-query adaptation and explore persistent test-time optimization (continual learning). In the current framework, model parameters are reset for each query, preventing knowledge accumulation over time. Allowing the model to update parameters across queries could enable incremental specialization to recurring patterns and improve performance in specific domains.

However, this direction introduces challenges such as error accumulation and model drift, especially under noisy supervision. Despite these challenges, the consistent improvements achieved by QueST suggest that query-driven adaptation is a promising foundation for progressively adaptive models.

Broader Impact. QueST moves LLMs from static inference toward adaptive inference, loosely resembling how humans refine their reasoning during a test by using the current problem to infer related patterns. This may enable more flexible and cost-efficient AI systems without external data or full retraining. At the same time, as in human problem solving, adaptation can fail or even reinforce mistakes when the query is ambiguous, misleading, or based on false premises, requiring safeguards against unreliable supervision and error accumulation.

6 Conclusion

In this work, we propose Query-Conditioned Test-Time Self-Training (QueST), a framework grounded in the observation that user queries themselves contain rich implicit signals for test-time adaptation. Instead of relying on external data or purely self-supervised signals, QueST constructs supervision directly from the input query by generating auxiliary problem–solution pairs, and performs efficient parameter adaptation via LoRA at test time. Across seven mathematical reasoning benchmarks and the GPQA-Diamond scientific reasoning benchmark, QueST consistently outperforms base models and existing test-time optimization baselines based on entropy or perplexity minimization, achieving state-of-the-art performance. Beyond performance, QueST highlights a new perspective on test-time optimization: rather than merely adapting representations, it enables models to actively acquire useful knowledge during inference. We hope this work opens up new directions for test-time learning in large language models, particularly toward more robust and knowledge-driven adaptation.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [3] Guhao Feng, Shengjie Luo, Kai Hua, Ge Zhang, Wenhao Huang, Di He, and Tianle Cai. In-place test-time training. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2604.06169>. Oral Presentation.
- [4] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [5] Moritz Hardt and Yu Sun. Test-time training on nearest neighbors for large language models. *arXiv preprint arXiv:2305.18466*, 2023.
- [6] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, et al. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3828–3850, 2024.
- [7] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- [8] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- [9] Jinwu Hu, Zhitian Zhang, Guohao Chen, Xutao Wen, Chao Shuai, Wei Luo, Bin Xiao, Yuanqing Li, and Mingkui Tan. Test-time learning for large language models. *arXiv preprint arXiv:2505.20633*, 2025.
- [10] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiabin Huang, Haitao Mi, and Dong Yu. R-zero: Self-evolving reasoning llm from zero data. *arXiv preprint arXiv:2508.05004*, 2025.
- [11] Srinivasan Iyer, Xi Victoria Lin, Ramakanth Pasunuru, Todor Mihaylov, Daniel Simig, Ping Yu, Kurt Shuster, Tianlu Wang, Qing Liu, Punit Singh Koura, et al. Opt-impl: Scaling language model instruction meta learning through the lens of generalization. *arXiv preprint arXiv:2212.12017*, 2022.
- [12] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022.
- [13] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *Advances in neural information processing systems*, 35:3843–3857, 2022.
- [14] Yafu Li, Xuyang Hu, Xiaoye Qu, Linjie Li, and Yu Cheng. Test-time preference optimization: On-the-fly alignment via iterative textual feedback. *arXiv preprint arXiv:2501.12895*, 2025.
- [15] Jian Liang, Ran He, and Tieniu Tan. A comprehensive survey on test-time adaptation under distribution shifts. *International Journal of Computer Vision*, 133(1):31–64, 2025.

- [16] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The twelfth international conference on learning representations*, 2023.
- [17] Bo Liu, Chuanyang Jin, Seungone Kim, Weizhe Yuan, Wenting Zhao, Ilia Kulikov, Xian Li, Sainbayar Sukhbaatar, Jack Lanchantin, and Jason Weston. Spice: Self-play in corpus environments improves reasoning. *arXiv preprint arXiv:2510.24684*, 2025.
- [18] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- [19] Mathematical Association of America. MAA Invitational Competitions: American Invitational Mathematics Examination. <https://maa.org/maa-invitational-competitions/>, 2026. Accessed: 2026-05-05.
- [20] Mathematical Association of America. American Mathematics Competitions. <https://maa.org/student-programs/amc/>, 2026. Accessed: 2026-05-05.
- [21] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori B Hashimoto. sl: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20286–20332, 2025.
- [22] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [23] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.
- [24] Minseok Seo, Mark Hamilton, and Changick Kim. Upsample anything: A simple and hard to beat baseline for feature upsampling. *arXiv preprint arXiv:2511.16301*, 2025.
- [25] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [26] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.
- [27] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- [28] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- [29] Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [30] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. *Advances in neural information processing systems*, 33:3008–3021, 2020.

- [31] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [32] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. *arXiv preprint arXiv:2006.10726*, 2020.
- [33] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- [34] Yiming Wang, Pei Zhang, Siyuan Huang, Baosong Yang, Zhuosheng Zhang, Fei Huang, and Rui Wang. Sampling-efficient test-time scaling: Self-estimating the best-of-n sampling in early decoding. *arXiv preprint arXiv:2503.01422*, 2025.
- [35] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 13484–13508, 2023.
- [36] Zengzhi Wang, Fan Zhou, Xuefeng Li, and Pengfei Liu. Octothinker: Mid-training incentivizes reinforcement learning scaling. *arXiv preprint arXiv:2506.20512*, 2025.
- [37] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [38] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [39] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- [40] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.
- [41] Cai Zhou, Zekai Wang, Menghua Wu, Qianyu Julie Zhu, Flora C Shi, Chenyu Wang, Ashia Wilson, Tommi Jaakkola, and Stephen Bates. Online reasoning calibration: Test-time training enables generalizable conformal llm reasoning. *arXiv preprint arXiv:2604.01170*, 2026.
- [42] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *The eleventh international conference on learning representations*, 2022.

A Implementation Details

A.1 Detailed Hyperparameters

Table 4 reports the complete set of hyperparameters used in our QueST experiments. We use the same configuration across all six model architectures (Qwen3-4B-Base, Qwen3-8B-Base, Qwen3-4B, Qwen3-8B, OctoThinker-3B, OctoThinker-8B) without per-model tuning.

Table 4: Detailed hyperparameters of QueST.

Hyperparameter	Value
<i>LoRA configuration</i>	
Rank (r)	16
Alpha (α)	32
Dropout	0.05
Target modules	q_proj, k_proj, v_proj, o_proj
Bias	none
<i>Test-time optimization</i>	
Optimizer	AdamW
Learning rate	1×10^{-4}
Number of optimization steps (T)	10 (default) / 40 (AIME24, AIME25)
Batch size	1
Gradient accumulation steps	4
Effective batch size	4
Max sequence length	4096
Precision	bf16
Loss	cross-entropy (next-token prediction)
<i>Auxiliary problem generation</i>	
Number of pairs (N)	5
Max new tokens	4096
Generator model p_ϕ	same as base model f_θ
<i>Final response inference</i>	
Decoding	greedy (do_sample=False)
Max new tokens	4096

B Qualitative Analysis of Query-adaptive Supervision

QueST applies the same generator, prompt, and sampling temperature to every input query. We examine whether, under this fixed configuration, the generated auxiliary supervision reflects the mathematical structure of the input query. We illustrate this with two MATH500 examples that differ in how easily they admit equivalent reformulations. The first is a coordinate-conversion query, whose generated variants remain within a single canonical template. The second is a complex-modulus query that can be reformulated as lattice-point counting in a Euclidean disk, allowing the generated variants to span multiple mathematical descriptions.

Template-bound Supervision. The first query asks for the polar coordinates of $(0, 3)$. The reasoning required is a two-step lookup: read $r = |y|$ and assign θ from the axis on which the point lies. The auxiliary problems QueST generates remain entirely within this template (Figure 5, top). One variant is a near-duplicate of the original query, while the other substitutes a different axis-aligned point. Each variant requires identical reasoning steps, and the supervision is highly redundant.

Question 0 (MATH500)
Q. Convert the point $(0,3)$ in rectangular coordinates to polar coordinates. Enter your answer in the form (r,θ) , where $r > 0$ and $0 \leq \theta < 2\pi$.
Generated Question
Q1. Convert the point $(0,3)$ in rectangular coordinates to polar coordinates. Enter your answer in the form (r,θ) , where $r > 0$ and $0 \leq \theta < 2\pi$.
Q2. Convert the point $(-4,0)$ in rectangular coordinates to polar coordinates. Enter your answer in the form (r,θ) , where $r > 0$ and $0 \leq \theta < 2\pi$.
Question 118 (MATH500)
Q. Find the number of ordered pairs (a,b) of integers such that $ a + bi \leq 5$.
Generated Question
Q1. Count the number of lattice points (x,y) in the coordinate plane that lie on or inside the circle centered at the origin with radius $\sqrt{8}$.
Q2. How many Gaussian integers of the form $u+vi$, where u and v are integers, satisfy $ u+vi < 5$?

Figure 5: Comparison of auxiliary supervision generated by QueST for two MATH500 queries under the same generator, prompt, and sampling temperature. Top (Question 0): all variants stay within a single canonical template, and one is a near-duplicate of the original query. Bottom (Question 118): the variants reframe the same lattice-point task in two different mathematical languages: a Cartesian circle and Gaussian integers.

Framework-spanning Supervision. The second query asks for the number of integer pairs (a, b) satisfying $|a + bi| \leq 5$, framed in the language of complex moduli. The underlying mathematical task is counting lattice points within a Euclidean disk, which admits multiple equivalent framings, and the auxiliary problems QueST generates span these framings (Figure 5, bottom). One variant restates the task in Cartesian terms (lattice points inside a circle of a given radius), and another invokes the Gaussian-integer framework explicitly. Both variants require the same underlying counting argument, yet expose the adapted model to qualitatively different mathematical vocabularies and solution narratives.

Interpretation. Holding the generator fixed, the framing diversity of the auxiliary supervision tracks the structural richness of the input query. For a query whose reasoning admits a single canonical template, the supervision collapses onto that template and occasionally produces near-duplicates of the query itself. For a query whose underlying task admits multiple equivalent framings, the supervision spans those framings naturally, exposing the adapted model to several mathematical languages of the same problem. This provides qualitative evidence that the input query encodes a latent signal not only about *what* reasoning is required but also about *which framings* of that reasoning are accessible. QueST inherits this signal without any explicit framing conditioning.

C Failure Case Analysis

We analyze cases where QueST fails to improve over the base model, including cases where test-time adaptation degrades performance. These failures show that effective adaptation depends not only on generating auxiliary supervision, but also on whether that supervision captures the reasoning structure needed by the input query. We identify two representative failure modes: surface-level matching and procedural retry without self-correction.

Surface-level Matching. The first failure mode arises when the generator matches the input query at the surface level, such as wording, numerical values, or syntactic form, without capturing the reasoning structure required to solve it. In this case, the auxiliary problems become numerical substitutions or trivial rephrasings of the input. Although they appear relevant, they provide little supervision for the reasoning pattern that the original query requires.

Figure 6 illustrates this on a parenthesization-counting problem from MATH500. Although the five auxiliary problems happen to share the input’s ground-truth answer of 4, they follow a nearly identical syntactic template. As a result, they do not teach the enumeration logic required by the original query, and QueST inherits the baseline’s incorrect answer 2.

Question 9 (MATH500)	
The expression $2 \cdot 3 \cdot 4 \cdot 5 + 1$ is equal to 121, since multiplication is carried out before addition. However, we can obtain values other than 121 for this expression if we are allowed to change it by inserting parentheses. For example, we can obtain 144 by writing $(2 \cdot (3 \cdot 4)) \cdot (5 + 1) = 144$. In total, how many values can be obtained from the expression $2 \cdot 3 \cdot 4 \cdot 5 + 1$ by inserting parentheses? (Note that rearranging terms is not allowed, only inserting parentheses). $\Rightarrow GT: 4$	
Baseline Answer	QueST Answer
To determine how many distinct values can be obtained from the expression $2 \cdot 3 \cdot 4 \cdot 5 + 1$ by inserting parentheses, we need to consider all possible ways to group the multiplication and addition operations. [...] Let's evaluate each of the above groupings: 1. $((2 \cdot (3 \cdot (4 \cdot 5))) + 1) = (2 \cdot (3 \cdot 20)) + 1 = (2 \cdot 60) + 1 = 120 + 1 = 121$ 5. $((2 \cdot 3) \cdot (4 \cdot (5 + 1))) = 6 \cdot (4 \cdot 6) = 6 \cdot 24 = 144$ [...] [boxed{2}]	We must evaluate all possible values from the fixed expression $2 \cdot 3 \cdot 4 \cdot 5 + 1$ by only inserting parentheses, without rearranging terms. [...] Total values obtained: $((2 \cdot 3) \cdot 4) \cdot (5 + 1) = 144$ $((2 \cdot 3) \cdot 4) \cdot 5 + 1 = 121$ $(2 \cdot (3 \cdot 4 \cdot 5 + 1)) = 121$ [...] Collecting distinct values gives {121, 144}. So there are 2 distinct values. Final Answer: boxed{2}
Generated Question $\Rightarrow$ All Generated Questions Converge to 4 (same as Question 9)	
Q1. How many distinct values can be obtained from the expression $3 \cdot 5 \cdot 2 \cdot 6 - 1$ by inserting parentheses?	
Q2. How many distinct values can be obtained from the expression $4 \cdot 2 \cdot 3 \cdot 7 + 2$ by inserting parentheses?	
Q3. How many distinct values can be obtained from the expression $6 \cdot 2 \cdot 5 \cdot 3 + 4$ by inserting parentheses?	
Q4. How many distinct values can be obtained from the expression $7 \cdot 3 \cdot 2 \cdot 4 - 2$ by inserting parentheses?	
Q5. How many distinct values can be obtained from the expression $8 \cdot 2 \cdot 3 \cdot 5 + 3$ by inserting parentheses?	

Figure 6: Surface-level matching failure on a parenthesization-counting query from MATH500. The generated auxiliary problems share a near-identical syntactic template and coincidentally converge to the same answer as the original query. However, they fail to cover the enumeration logic required by the input, and the adapted model inherits the baseline’s incorrect answer.

Procedural Retry without Self-Correction. A second failure mode arises when the adapted model encounters its own computational error and attempts to recover. Because QueST’s auxiliary supervision contains only successful solution traces, the adapted model learns to imitate the step-by-step procedural format of the supervision but is not exposed to examples of error diagnosis or self-correction.

When an error occurs at inference, the model can recognize that something is wrong, e.g., “ $\cos\theta$ cannot be less than -1 , so there must be a mistake.” However, it lacks a learned mechanism to identify and revise the faulty step. Instead, it opens a new numbered step and reproduces the same erroneous calculation.

Figure 7 illustrates this behavior on a vector-angle problem: the adapted model recognizes the impossibility of its own intermediate result yet cannot localize the faulty step, and instead opens a new numbered step that reproduces the same erroneous block, resulting in a procedural retry rather than a substantive correction.

Implications. These two failure modes highlight two distinct limitations of the generated supervision: insufficient structural diversity and the absence of error-correction traces. *Surface-level matching* reflects a limitation in *what the supervision contains*: the generated variants are aligned with the input at the surface level but do not cover the required reasoning structure. In contrast, *proce-*

Question 260 (MATH500)
Let $\mathbf{A} = (1, -11, 2)$, $\mathbf{B} = (3, -4, 1)$, and $\mathbf{C} = (-2, 1, -1)$. Compute $\angle ABC$ in degrees.
QueST Answer
Step 1: To find $\angle ABC$, use the vectors from $\mathbf{B}$ to the other points: $\vec{BA} = \mathbf{A} - \mathbf{B} = (1-3, -11-(-4), 2-1) = (-2, -7, 1)$, $\vec{BC} = \mathbf{C} - \mathbf{B} = (-2-3, 1-(-4), -1-1) = (-5, 5, -2)$. Step 4: Correct the vectors: $\vec{BA} = \mathbf{A} - \mathbf{B} = (1-3, -11-(-4), 2-1) = (-2, -7, 1)$, $\vec{BC} = \mathbf{C} - \mathbf{B} = (-2-3, 1-(-4), -1-1) = (-5, 5, -2)$. Their dot product is $\vec{BA} \cdot \vec{BC} = (-2)(-5) + (-7)(5) + (1)(-2) = 10 - 35 - 2 = -27$. Their magnitudes are $\vec{BA} = \sqrt{(-2)^2 + (-7)^2 + 1^2} = \sqrt{54} = 3\sqrt{6}$, $\vec{BC} = \sqrt{(-5)^2 + 5^2 + (-2)^2} = \sqrt{54} = 3\sqrt{6}$. So $\cos \theta = \frac{-27}{(3\sqrt{6})(3\sqrt{6})} = \frac{-27}{18} = -\frac{3}{2}$. This is impossible, so the vectors are not correctly defined. Step 5: The correct vectors are: $\vec{BA} = \mathbf{A} - \mathbf{B} = (1-3, -11-(-4), 2-1) = (-2, -7, 1)$, $\vec{BC} = \mathbf{C} - \mathbf{B} = (-2-3, 1-(-4), -1-1) = (-5, 5, -2)$. Their dot product is $\vec{BA} \cdot \vec{BC} = (-2)(-5) + (-7)(5) + (1)(-2) = 10 - 35 - 2 = -27$. Their magnitudes are $\vec{BA} = \sqrt{(-2)^2 + (-7)^2 + 1^2} = \sqrt{54} = 3\sqrt{6}$, $\vec{BC} = \sqrt{(-5)^2 + 5^2 + (-2)^2} = \sqrt{54} = 3\sqrt{6}$. So $\cos \theta = \frac{-27}{(3\sqrt{6})(3\sqrt{6})} = \frac{-27}{18} = -\frac{3}{2}$. This is impossible, so the vectors are not correctly defined. Step 10: The correct vectors are: $\vec{BA} = \mathbf{A} - \mathbf{B} = (1-3, -11-(-4), 2-1) = (-2, -7, 1)$, $\vec{BC} = \mathbf{C} - \mathbf{B} = (-2-3, 1-(-4), -1-1) = (-5, 5, -2)$. Their dot product is $\vec{BA} \cdot \vec{BC} = (-2)(-5) + (-7)(5) + (1)(-2) = 10 - 35 - 2 = -27$. Their magnitudes are $\vec{BA} = \sqrt{(-2)^2 + (-7)^2 + 1^2} = \sqrt{54} = 3\sqrt{6}$, $\vec{BC} = \sqrt{(-5)^2 + 5^2 + (-2)^2} = \sqrt{54} = 3\sqrt{6}$. So $\cos \theta = \frac{-27}{(3\sqrt{6})(3\sqrt{6})} = \frac{-27}{18} = -\frac{3}{2}$. This is impossible, so the vectors are not correctly defined. Final Answer: $\boxed{180^\circ}$
Base Answer
To find the angle $\angle ABC$, we can use the dot product formula for vectors. First, we need to find the vectors $\vec{BA}$ and $\vec{BC}$. Now, we can find the angle $\angle ABC$ using the inverse cosine function: $\angle ABC = \cos^{-1}\left(-\frac{3}{2}\right) = 120^\circ$ So, the angle $\angle ABC$ is $\boxed{120^\circ}$.

Figure 7: Procedural retry failure on a vector-angle query. The adapted model repeatedly reproduces the same erroneous calculation block, even after recognizing that the intermediate cosine value is impossible. This leads to repeated procedural restarts rather than localized self-correction, and the final answer becomes incorrect.

dural retry reflects a limitation in *what the supervision omits*: the supervision contains successful solution traces but lacks examples of error diagnosis and recovery. Both observations motivate future work on supervision generation that improves structural diversity and includes error-correction traces.

D Extended Ablation: Number of Generated Problems

In the main paper, we report results for $N \in \{1, 3, 4, 5\}$ generated auxiliary problem–solution pairs (Table 3, Middle). Here, we extend this ablation to larger values of $N \in \{6, 7, 8\}$ to examine whether additional supervision continues to provide gains beyond $N = 5$ and to characterize the resulting accuracy–efficiency trade-off.

All experiments are conducted on Qwen3-4B-Base, evaluated on MATH500 with the same hyperparameters as the main experiments. For each query, we generate N auxiliary problem–solution pairs and perform test-time optimization on this expanded supervision set.

Table 5: Effect of the number of generated problems on Qwen3-4B-Base evaluated on MATH500. Although accuracy continues to improve with larger N , we use $N = 5$ in our main experiments to balance performance with the linearly growing test-time compute.

#Problems	Acc
1	57.80
3	63.00
4	64.20
5	68.60
6	69.20
7	70.00
8	70.40

Table 5 shows that QueST’s performance continues to scale with N , improving from 68.60 at $N=5$ to 70.40 at $N=8$. The marginal gain per additional pair, however, diminishes from $+4.4$ ($N:4 \rightarrow 5$) to $+0.4$ ($N:7 \rightarrow 8$), indicating clear diminishing returns. Since per-query compute grows linearly with N , the choice of N effectively controls the accuracy–efficiency trade-off. We adopt $N = 5$ as the default operating point in our main experiments, as it captures most of the achievable accuracy at a substantially lower compute budget.

E Licenses for Existing Assets

We list the licenses and sources of all existing assets (models, datasets, and software libraries) used in this work. All assets are used for academic research and evaluation in accordance with their respective licenses, access conditions, and terms of use. For benchmarks based on AMC and AIME problems, we use publicly accessible past competition materials only for evaluation and do not redistribute the original problem sets.

Table 6: Licenses and sources of all existing assets used in this work.

Asset	License	Source
<i>Pre-trained Models</i>		
Qwen3-4B [38]	Apache 2.0	https://huggingface.co/Qwen/Qwen3-4B
Qwen3-8B [38]	Apache 2.0	https://huggingface.co/Qwen/Qwen3-8B
Qwen3-4B-Base [38]	Apache 2.0	https://huggingface.co/Qwen/Qwen3-4B-Base
Qwen3-8B-Base [38]	Apache 2.0	https://huggingface.co/Qwen/Qwen3-8B-Base
OctoThinker-3B [36]	Llama 3.2 Community License	https://huggingface.co/OctoThinker/OctoThinker-3B-Hybrid-Base
OctoThinker-8B [36]	Llama 3.2 Community License	https://huggingface.co/OctoThinker/OctoThinker-8B-Hybrid-Base
<i>Evaluation Benchmarks</i>		
MATH500 [7, 16]	MIT	https://huggingface.co/datasets/HuggingFaceH4/MATH-500
GSM8K [2]	MIT	https://huggingface.co/datasets/openai/gsm8k
OlympiadBench [6]	MIT	https://huggingface.co/datasets/zwe99/simplerl-olympiadBench
GPQA-Diamond [23]	CC BY 4.0	https://huggingface.co/datasets/Idavidrein/gpqa
Minerva [13]	License not specified	https://huggingface.co/datasets/zwe99/simplerl-minerva-math
AMC [20]	MAA copyrighted (academic use)	https://huggingface.co/datasets/zwe99/amc23
AIME24 [19]	MAA copyrighted (academic use)	https://huggingface.co/datasets/HuggingFaceH4/aime_2024
AIME25 [19]	MAA copyrighted (academic use)	https://huggingface.co/datasets/yentinglin/aime_2025
<i>Software Libraries</i>		
PyTorch	BSD-3-Clause	https://github.com/pytorch/pytorch
Hugging Face Transformers	Apache 2.0	https://github.com/huggingface/transformers
PEFT (LoRA) [8]	Apache 2.0	https://github.com/huggingface/peft
TENT [32]	MIT	https://github.com/DequanWang/tent
TLM [9]	Apache 2.0	https://github.com/Fhujinwu/TLM